

Clase 11: ALGORITMOS DE BÚSQUEDA

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria
Tolosa

Outline

1. Búsqueda secuencial
2. Búsqueda binaria
3. Ejercicios

A partir de ahora vamos a hacer (más) foco en:

- resolución de problemas (usando la computadora),
- abstraer y generalizar conceptos,
- concentrarnos en las ideas (que luego se convertirán en código),
- algorithmic thinking,

siempre con aplicaciones.

Ejercicio

Implementar la función `busqueda_secuencial` que toma una lista, un elemento (del tipo de la lista) y devuelve *True* si el elemento está en la lista y *False* en caso contrario.

```
def busqueda_secuencial(l, elem):  
    for x in l :  
        if x == elem:  
            return True  
    return False  
  
def main():  
    l = [15, 35, 40, 2, 34, 28, 50]  
    busqueda_secuencial(l, 35)
```

Algunas preguntas:

- Cuántos elementos tiene que revisar el algoritmo?
- Cuál es el *peor caso* de comparaciones que debe realizar?
- Si la lista *l* estuviera ordenada de menor a mayor, podemos reutilizar la función anterior? Podemos *aprovechar* esta propiedad?

BÚSQUEDA BINARIA

Tenemos la
lista l

l	[2, 15, 28, 34, 35, 40, 50]						
posición	0	1	2	3	4	5	6

y queremos ver si el elemento 35 está en la lista. Una forma alternativa es pensar el problema como tratar de obtener la posición en la que se encuentra (si es que está).

La idea considerar una **sucesión de intervalos**, cada vez **más chicos**, tales que si el número x (35) está en la lista entonces está contenido entre esos índices.

Representamos al intervalo como $[low, high]$

Inicialmente: el intervalo toma **toda la lista**. Es decir, $low = 0$, $high = 6$. Si el **35 está en l**, entonces **el índice del mismo está en ese intervalo**

Inicialmente: $low = 0$, $high = 6$. Si el 35 está en L , entonces el índice del mismo está en el intervalo $[low, high]$.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
	low						high

Ahora, partimos el intervalo a la mitad tomando el elemento ubicado en el medio, $mid = (0 + 6)/2 = 3$, y analizamos el elemento en esa posición.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
	low			mid			high

Nos hacemos dos preguntas:

- El elemento en la posición mid es el buscado? Si lo fuese, qué podemos decir?
- Si no es el buscado, en cuál intervalo podemos garantizar que *podría* estar? Por qué?

Correcto! $\text{low} = \text{mid} + 1 = 4$, $\text{high} = 6$. Podemos descartar la primera mitad de la lista. Si el 35 está en l , entonces el índice del mismo está en el intervalo $[\text{low}, \text{high}] = [4, 6]$.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
					low	high	

Aplicamos nuevamente el mismo razonamiento y partimos el intervalo a la mitad tomando el elemento ubicado en el medio, $\text{mid} = (4 + 6)/2 = 5$, y analizamos el elemento en esa posición

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
					low	mid	high

Nos hacemos dos preguntas:

- El elemento en la posición mid es el buscado? Si lo fuese, qué podemos decir?
- Si no es el buscado, en cuál intervalo podemos garantizar que *podría* estar?

Por qué?

Correcto! $low = 4$, $high = mid - 1 = 4$. Podemos descartar la segunda mitad de la lista. Si el 35 está en l , entonces el índice del mismo está en el intervalo $[low, high] = [4, 4]$.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
					low		
					mid		
					high		

Aplicamos nuevamente el mismo razonamiento y partimos el intervalo a la mitad tomando el elemento ubicado en el medio, $mid = (4 + 4)/2 = 4$, y analizamos el elemento en esa posición

- El elemento en la posición mid es el buscado?

Supongamos ahora que estamos buscando el número 36. Los pasos son exactamente los mismos hasta el punto de la slide anterior. Si el 36 está en l, entonces el índice del mismo está en el intervalo $[low, high] = [4, 4]$.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
				low			
				mid			
				high			

Aplicamos nuevamente el mismo razonamiento y partimos el intervalo a la mitad tomando el elemento ubicado en el medio, $mid = (4 + 4)/2 = 4$, y analizamos el elemento en esa posición

- El elemento en la posición mid es el buscado?

Supongamos ahora que estamos buscando el número 36. Los pasos son exactamente los mismos hasta el punto de la slide anterior.

Si el 36 está en l , entonces el índice del mismo está en el intervalo $[low, high] = [4, 4]$.

L	[2,	15,	28,	34,	35,	40,	50]
Posición	0	1	2	3	4	5	6
					low		
					mid		
					high		

Aplicamos nuevamente el mismo razonamiento y partimos el intervalo a la mitad tomando el elemento ubicado en el medio, $mid = (4 + 4)/2 = 4$, y analizamos el elemento en esa posición

- El elemento en la posición mid es el buscado? NO.

Dado que 36 es mayor a mid , entonces actualizamos el intervalo a $[low, high] = [mid+1, high] = [5, 4]$.

Búsqueda Binaria

Qué podemos deducir del intervalo $[5,4]$?

Y esto qué significa?

Cómo podemos justificar esto último?

Resumiendo:

- Si el elemento está, entonces en algún momento lo encontramos al tomar el punto medio **mid** y verificando el valor de la lista en esa posición.
- Si no está, en algún momento **low > high**.
- Cuidado: al tomar el punto medio, utilizar división entera ($//$) para evitar problemas en caso que el rango **[low,high]** tenga una cantidad par de posiciones (y, por lo tanto, no hay un único elemento en el medio).

Sabemos *a priori* cuántas iteraciones vamos a hacer en el peor de los casos?

Cómo sabemos cuándo vamos a terminar de iterar?

EJERCICIO



1. Implementar el algoritmo de *búsqueda secuencial*, que toma una lista (no necesariamente ordenada) l y un elemento x y devuelve True en caso de x esté en l , y False en caso contrario.
2. Implementar el algoritmo *búsqueda binaria* de forma iterativa, que toma una lista ordenada (en forma ascendente) l y un elemento x (del tipo de los elementos de la lista) y devuelve True en caso que x esté en l , y False en caso contrario.
3. Implementar el algoritmo de *búsqueda binaria* usando recursión.
Sugerencia, agregar parámetros adicionales a la función para indicar los límites low y high.

```
def busqueda_binaria_recursiva (l, x, low, high):  
    Pass
```

4. Ejercicio 10.10.

```
def busqueda_binaria ( l , t a r g e t ) :  
    # Inicializamos las variables del intervalo.  
    low = 0  
    high = len ( l ) - 1  
    while low <= high :  
        mid = low + int ( ( high - low ) / 2 )  
        if l [mid] == t a r g e t :  
            return True # Si es el mismo valor, lo encontramos .  
        elif l [mid] < t a r g e t :  
            low = mid + 1 # Descartamos valores menores a target  
        else:  
            high = mid - 1 # Descartamos valores mayores a target  
    return False # Termino el while , low > high . target no esta  
    en l .
```